SOLUCIONARIO OFICIAL: 30 CONSULTAS SQL

Manual de Respuestas y Resolución de Ejercicios | Base de Datos PostgreSQL

Bloque I: Respuestas a Consultas 1 a 10

1. Consulta SELECT básica

```
SELECT * FROM tab_prov;
```

2. Consulta SELECT con filtro WHERE

```
SELECT * FROM tab_prod WHERE ind_estado = true;
```

3. Consulta SELECT con ORDER BY

```
SELECT * FROM tab_prov ORDER BY nom_prov ASC;
```

4. Consulta SELECT con LIMIT

```
SELECT id_ciud, nom_ciud FROM tab_ciud LIMIT 10;
```

5. Consulta SELECT con COUNT

```
SELECT COUNT(*) AS total_ordenes FROM tab_orden_compra;
```

6. Consulta SELECT con SUM

```
SELECT SUM(val_neto) AS total_facturado FROM tab_det_compra;
```

7. Consulta SELECT con AVG

```
SELECT AVG(val_cupocre) AS promedio_cupo FROM tab_prov;
```

8. Consulta SELECT con GROUP BY

```
SELECT id_dpto, COUNT(*) AS total_ciudades FROM tab_ciud GROUP BY id_dpto;
```

9. Consulta INSERT

```
INSERT INTO tab_areas (id_area, nom_area, nom_ecarg) VALUES (9, 'Auditoría', 'Elena
Beltrán');
```

10. Consulta UPDATE

```
UPDATE tab_prov SET val_puntos = 98 WHERE id_prov = 1000000001;
```

Bloque II: Respuestas a Consultas 11 a 20

11. Consulta DELETE

```
DELETE FROM tab_orden_compra WHERE id_orden = 100030;
```

12. Consulta JOIN

```
SELECT c.nom_ciud, d.nom_dpto FROM tab_ciud c JOIN tab_dptos d ON c.id_dpto = d.id_dpto;
```

13. Consulta con Subconsulta

```
SELECT * FROM tab_prod WHERE id_prod IN (SELECT id_prod FROM tab_prod_prov WHERE val_prod >
2000000);
```

14. Consulta con CASE

```
SELECT nom_comp, val_cupocre,
       CASE
           WHEN val_cupocre >= 5000000 THEN 'Cupo Alto'
           WHEN val_cupocre BETWEEN 3000000 AND 4999999 THEN 'Cupo Medio'
           ELSE 'Cupo Bajo'
       END AS categoria_credito
FROM tab_prov;
```

15. Consulta con UNION

```
SELECT id_prod FROM tab_orden_compra UNION SELECT id_prod FROM tab_det_compra;
```

16. Consulta con UNION ALL

```
SELECT id_ciud FROM tab_prov UNION ALL SELECT id_ciud FROM tab_enc_compra;
```

17. Consulta con HAVING

```
SELECT id_factura, COUNT(*) AS lineas_detalle FROM tab_det_compra GROUP BY id_factura HAVING
COUNT(*) > 1;
```

18. Consulta con EXISTS

```
SELECT * FROM tab_prov p WHERE EXISTS (SELECT 1 FROM tab_enc_compra e WHERE p.id_prov =
e.id_prov);
```

19. Consulta con LIKE

```
SELECT * FROM tab_prov WHERE (datos_prov).correo LIKE '%tecnosum%';
```

20. Consulta con BETWEEN

```
SELECT * FROM tab_enc_compra WHERE fec_factura BETWEEN '2024-02-01' AND '2024-02-29';
```

Bloque III: Respuestas a Consultas 21 a 29

21. Verificación de Volumetría

```
SELECT 'Proveedores' AS tabla, COUNT(*) AS total FROM tab_prov
UNION ALL
SELECT 'Encabezado Compras', COUNT(*) FROM tab_enc_compra
UNION ALL
SELECT 'Detalle Compras', COUNT(*) FROM tab_det_compra
UNION ALL
SELECT 'Pagos', COUNT(*) FROM tab_pagos;
```

22. Verificación de anomalías (Espacios)

```
SELECT id_prod, nom_prod FROM tab_prod WHERE nom_prod LIKE ' %' OR nom_prod LIKE '% ';
```

23. Detección de Caracteres Extraños (Encoding)

```
SELECT id_factura, val_observa FROM tab_enc_compra WHERE val_observa LIKE '%%';
```

24. Auditoría General de Constraints

```
SELECT constraint_name AS "Nombre_Constraint", constraint_type AS "Tipo", table_name AS
"Tabla"
FROM information_schema.table_constraints
WHERE table_name = 'tab_det_compra';
```

25. Consulta de Tipo Compuesto (UDT)

```
SELECT nom_comp, (datos_prov).direccion AS direccion, (datos_prov).telefono AS telefono,
(datos_prov).correo AS correo FROM tab_prov;
```

26. Cálculo Financiero por Factura

```
SELECT id_factura, SUM(val_pago) AS total_abonado, MIN(val_saldo) AS saldo_final FROM
tab_pagos GROUP BY id_factura;
```

27. Cruce Multitabla con Filtros Texto

```
SELECT p.nom_comp, pr.nom_prod, pp.val_prod
FROM tab_prod_prov pp
JOIN tab_prov p ON pp.id_prov = p.id_prov
JOIN tab_prod pr ON pp.id_prod = pr.id_prod
WHERE pp.ind_disp = true AND (pr.nom_prod LIKE '%Gamer%' OR pr.nom_prod LIKE '%Oficina%');
```

28. Agregación Temporal Mensual

```
SELECT EXTRACT(MONTH FROM e.fec_factura) AS mes, COUNT(DISTINCT e.id_factura) AS
num_facturas, SUM(d.val_neto) AS total_mes
FROM tab_enc_compra e
JOIN tab_det_compra d ON e.id_factura = d.id_factura
GROUP BY EXTRACT(MONTH FROM e.fec_factura)
ORDER BY mes ASC;
```

29. Resumen de Órdenes por Área

```
SELECT a.nom_area, COUNT(o.id_orden) AS cantidad_ordenes, SUM(o.val_cant) AS unidades_pedidas
FROM tab_orden_compra o
JOIN tab_areas a ON o.id_area = a.id_area
GROUP BY a.nom_area;
```

Bloque IV: Consulta Compleja (Respuesta Ejercicio 30)

```
SELECT a.nom_area AS Area,
       COUNT(DISTINCT o.id_prod) AS Num_Productos_Diferentes,
       AVG(o.val_cant) AS Promedio_Cant_Por_Orden,
       SUM(d.val_neto) AS Inversion_Total_Area
FROM tab_areas a
-- 1. Unir Áreas con Órdenes de Compra
INNER JOIN tab_orden_compra o ON a.id_area = o.id_area
-- 2. Unir Órdenes con Detalles de Compra por Producto
INNER JOIN tab_det_compra d ON o.id_prod = d.id_prod
-- 3. Filtrar los productos activos de alto valor
WHERE d.val_neto > 500000
  AND o.val_cant >= 5
-- 4. Agrupar los resultados por Área
GROUP BY a.nom_area
-- 5. Filtrar los grupos que cumplen la condición
HAVING COUNT(DISTINCT o.id_prod) >= 1
-- 6. Ordenar por la Inversión Total Descendente
ORDER BY Inversion_Total_Area DESC;
```

Respuestas a las Preguntas Teórico-Prácticas

Pregunta 1: Respuesta B (GROUP BY y HAVING)
Explicación: Para filtrar datos agregados (`COUNT(*) > 10`) tras agrupar por proyecto, es obligatorio usar `GROUP BY` junto a la cláusula `HAVING`.

Pregunta 2: Respuesta B (JOIN con condición ON)
Explicación: La cláusula `JOIN` con su condición `ON` es la construcción estándar relacional más eficiente para combinar registros cruzados entre la tabla de empleados y departamentos.

Pregunta 3: Respuesta C (Expresión CASE)
Explicación: La expresión `CASE` permite evaluar lógica condicional (`IF-THEN-ELSE`) directamente dentro de la lista de proyección del `SELECT` para categorizar registros dinámicamente.